

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_7049e1ed-b93\agent-a082a91af7782d1d3.jsonl`  
- SHA-256: `3357663bad091b66d1146b1b2f266e9c1055238852487cc3adc340cfb0d86ad`  
- Source modified: `2026-07-02T04:58:22+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_7049e1ed-b93`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T04:56:52.330Z)

You are an adversarial code reviewer. Today is 2026-07-02. Review PR #418 in the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (gh pr view 418, gh pr diff 418; the working copy is in worktree C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-source-cache ? read code there for context).

The PR claims to fix: R1-03 (/api/source full re-download per Range request) (evidence in C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md ? Grep for the IDs; honor the verifier notes' scoping corrections). Hunt REAL defects: correctness bugs in the diff, broken/vacuous tests, behavior changes beyond the brief, staging accidents (files that should not be in the diff ? check the file list!), convention violations (default-ON when brief says OFF, missing schema-version check, CWD assumptions), and whether the fix actually addresses the finding. READ-ONLY ? do not modify or comment on GitHub. blocker = must not merge; major = fix before merge; minor = nice-to-have.

2. user (2026-07-02T04:56:59.646Z)

```
title: fix(api): /api/source downloads a remote source ONCE into a reusable cache (R1-03)  
state: OPEN  
author: avidullu (Avi)  
labels:  
assignees:  
reviewers:  
projects:  
milestone:  
number: 418  
url: https://github.com/Khelsutra/badminton-highlight-indexer/pull/418  
additions: 225  
deletions: 9  
auto-merge: disabled  
--
```

What

Fixes audit finding ****R1-03****: `GET /api/source/{video\_id}` called `storage.acquire\_local\_input` on **every** request, and for a REMOTE-backed video (`gs://` / `s3://` / `gdrive://`) that meant a fresh `tempfile.mkdtemp(prefix="rally-input-")` + a **full object download per request** ? never deleted. `FileResponse` serves Range, so one browser scrubbing a `` tag issues many independent requests to the same URL: ~10 seeks on a 4 GB match ? ~40 GB of bucket egress ****and**** ~40 GB of orphaned temp dirs, with every seek stalling behind a multi-GB download.

Now (`backend/main.py::\_acquire\_source\_for\_serving`):

- A remote source is fetched **once** into `/source\_cache/<video\_id>/<name>` and every

later request streams that local copy.

- **Atomic publish**: the download goes to a unique `<name>.<uuid>.partial` in the cache dir, then `os.replace` onto the served name ? a truncated file can never sit at the served path, and a failed fetch cleans its partial (502 surfaced, retry starts clean). No `finally`-delete of the served file ? `FileResponse` streams after the handler returns.
- **Concurrent first requests** serialize on a per-video lock; losers re-check under the lock and reuse the winner's copy (one download total).
- The download is **free-space guarded** (`require_free_bytes`, D11 / #407) when the backend can `stat` the object size ? refused with 507 *before* any bytes move; the cache lives under the guarded output root and is a regenerable intermediate (safe to delete any time).
- **Local-path videos** stay a byte-identical identity passthrough ? no copy, no cache dir, `storage.fetch` never called; path-safety unchanged (`safe_output_filename` on the video_id path component + a sanitized cached filename).
- **Proxy warming** now covers remote sources too (the cached copy is a plain local file), closing the "remote-source proxying is a follow-up" note from #406 ? once the D4 proxy exists it wins as before.

Why

Direct cloud-egress cost leak + a disk-fill denial-of-service of the appliance, reachable through a supported flow (remote-registered assets via P4 `/api/assets` + the P8 annotate-in-UI player). See `AUDIT_REPORT_khelsutra-guru_2026-07-02.md` R1-03 evidence + verifier notes.

Verification

Ran locally (Windows, Python 3.13):

- `python -m pytest tests/test_api_annotations.py -q` ? **12 passed** (6 new tests: remote fetched once across two requests with Range served from cache; local identity passthrough with fetch forbidden; stale `.partial` never served; failed fetch ? 502 and no partial left; 507 free-space refusal before fetch; two-thread cold race downloads once).
- `python -m pytest tests/test_input_source.py tests/test_api_cloud_dispatch.py tests/test_storage.py -q` ? **81 passed** (adjacent storage/input behavior unchanged).
- `ruff check backend/main.py tests/test_api_annotations.py` ? clean.
- `python -m mypy backend/main.py tests/test_api_annotations.py` ? no issues (note `backend.main` is mypy-quarantined via `mypy.ini`).

Not verified locally: a real bucket download (network is mocked per repo test policy ? offline suite only); full suite + coverage floor is CI's job.

? Generated with [Claude Code](https://claude.com/claude-code)

3. user (2026-07-02T04:57:01.177Z)

55-[Omitted long context line]

56-- why it matters: This is the last HIGH-severity item of the cleanup audit (P10, the only non-Phase-3 ? row in its \$6 tracker) and it is compounding: every new endpoint PR lands in the god-module, widening the conflict surface and making the split more expensive each week. The 3-file extraction in #390 is now unmergeable.

57-- proposed fix: Close #390 and re-cut the orchestration.py extraction fresh from origin/master (small, ~1 session), or rebase it now. Then continue the split (CLI handlers -> backend/cli/, new /api/source|api/assets|api/capabilities routes -> backend/api/) before more endpoints accrete. Update the audit doc's stale '~1450 lines' sizing while at it.

58-[Omitted long context line]

59-[Omitted long context line]

60-

61-### R1-02 [P1/M] critical-fix ? badminton-highlight-indexer

62-**api/process** re-runs the paid Gemini segmenter for an already-processed video (re-bill on refresh/'Try again') ? still unfixed at HEAD

63-- anchor: `backend/main.py:1172` · tracked in: Khelsutra/badminton-highlight-indexer#340 (also #330) · finder: engine:correctness

64-- evidence: `backend/main.py:1171-1172` computes `video_id` and `was_reingest = db.get_video(video_id)` is not None, but `was_reingest` is only used for the report/destroy-after-confirm; there is NO reuse/skip branch ? the segmenter always runs (`main.py:1251-1255`). Verified against gh: issue #340 OPEN ('re-runs the full (paid) segmenter anyway'), plus #330 (MD5

```

segmentation reuse) OPEN.
65-- why it matters: A page refresh or SPA 'Try again' on an already-indexed video (same MD5)
re-bills Gemini (~$0.10/8-min match) and burns a multi-minute compute run for zero new
information. Live-triggered once in the alpha (2026-06-22, per #340).
66-- proposed fix: In _execute_processing, when db.get_video(video_id) exists with
status='processed' and existing play_segments (and the request doesn't explicitly force
reprocess), return the stored segments as the job result instead of dispatching the segmenter;
add a `force` flag to ProcessRequest for deliberate re-ingest.
67-[Omitted long context line]
68-[Omitted long context line]
69-
70:### R1-03 [P1/M] critical-fix ? badminton-highlight-indexer
71-*/api/source re-downloads the ENTIRE remote video from the bucket on every HTTP request
(each browser Range request) into a fresh, never-deleted temp dir**
72-- anchor: `backend/main.py:646` · tracked in: ? · finder: engine:correctness
73-[Omitted long context line]
74-- why it matters: Direct cloud-egress cost leak plus a disk-fill denial-of-service of the
appliance, reachable through a supported flow (remote-registered assets + P8 annotate-in-UI).
75-- proposed fix: Cache the fetched local copy keyed by video_id (e.g. under <output>/proxies
or a per-video workspace source/ dir) and reuse it across requests; extend proxy warming to
remote sources; delete or reuse the scratch dir instead of mkdtemp-per-request.
76-[Omitted long context line]
77-[Omitted long context line]
78-
79:### R1-04 [P1/M] tech-debt ? badminton-highlight-indexer
80-**Import cycles: extracted API modules reach back into backend.main globals at call time;
config layer imports pipeline layer**
81-- anchor: `backend/api/job_worker.py:75` · tracked in: docs/CODE_CLEANUP_AUDIT_2026-06-24.md
P5 note + D1/P10 follow-up (line 221) · finder: engine:structure
82-[Omitted long context line]
83-- why it matters:
84-- proposed fix: Kill the shim as the concrete step-2 of D1: move db_instance/global_config
onto app.state (create_app already exists per P5) and pass config/db explicitly to
job_worker/uploads, then lift backend.main and backend.worker.__main__ out of mypy quarantine.
For the layering inversion, move the segmenter-name validation out of the pydantic validator
into a startup check (backend/config/startup.py) so config/ no longer imports pipeline/.
85-[Omitted long context line]

```

4. user (2026-07-02T04:57:07.667Z)

```

diff --git a/backend/main.py b/backend/main.py
index 99246fe..078fa2b 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -620,13 +620,79 @@ def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
     _proxy_inflight.discard(video_id)

+# Per-video locks so concurrent FIRST requests for a cold remote source serialize on ONE
download
+# (the losers re-check and reuse the winner's copy) instead of each pulling the full object.
+_source_fetch_locks: "dict[str, threading.Lock]" = {}
+_source_fetch_locks_guard = threading.Lock()
+
+
+def _source_cache_dir(cfg) -> str:
+    """Where /api/source caches fetched REMOTE sources ? under the guarded local output
root."""
+    return os.path.join(output_base(cfg), "source_cache")
+
+
+def _acquire_source_for_serving(video_id: str, raw_ref: str, cfg) -> str:
+    """Resolve a video's recorded filepath to the LOCAL file ``/api/source`` streams.
+
+    A local path is an identity passthrough (no copy, no temp ? byte-identical to serving the

```

```

+ recorded path directly). A REMOTE ref (``gs://``/``s3://``/``gdrive://``?) is fetched ONCE
into
+ ``<output>/source_cache/<video_id>/<name>`` and REUSED: ``FileResponse`` serves Range, so
one
+ scrubbing ``<video>`` tag issues many independent requests to this URL ? before this cache
each
+ request re-downloaded the full object into a fresh, never-deleted temp dir (a cloud-egress
bill
+ plus a disk-fill of the appliance; audit R1-03).
+
+ Concurrency + crash safety: the download goes to a unique ``*.partial`` name inside the
cache
+ dir and is published with an atomic ``os.replace`` ? a truncated file can never sit at the
+ served name, and a failed fetch leaves nothing behind. Racing first requests serialize on a
+ per-video lock. The download is free-space guarded (D11) when the backend can ``stat`` the
+ object's size; the cached copy is a regenerable intermediate (safe to delete any time)."""
+ settings = _storage_settings(cfg)
+ ref = storage.resolve_input_ref(raw_ref, settings)
+ if ref.backend == "local":
+     return ref.key # identity ? today's local behaviour, byte-for-byte
+
+ vid_dir = os.path.join(_source_cache_dir(cfg), safe_output_filename(video_id))
+ try:
+     fname = safe_output_filename(ref.name or "in.mp4")
+ except ValueError: # a hostile/odd remote object name never dictates the local filename
+     fname = "in.mp4"
+ final = os.path.join(vid_dir, fname)
+ if os.path.isfile(final):
+     return final # cache hit ? no re-download, whatever the browser's Range pattern
+
+ with _source_fetch_locks_guard:
+     lock = _source_fetch_locks.setdefault(video_id, threading.Lock())
+ with lock:
+     if os.path.isfile(final):
+         return final # a concurrent first request fetched it while we waited on the lock
+     os.makedirs(vid_dir, exist_ok=True)
+     need = None
+     try: # sizing is best-effort ? a backend that can't stat still fetches (and fails
loudly)
+         need = int(storage.get_storage(ref.backend, config=settings).stat(ref).size)
+     except Exception: # noqa: BLE001 - stat is optional; require_free_bytes(None) is a no-
op
+         need = None
+     require_free_bytes(vid_dir, need, stage="source-cache download")
+     tmp = os.path.join(vid_dir, f"{fname}.{uuid.uuid4().hex}.partial")
+     try:
+         storage.fetch(ref.uri, tmp, config=settings)
+         os.replace(tmp, final) # atomic publish ? readers only ever see a COMPLETE file
+     finally:
+         if os.path.exists(tmp): # fetch (or publish) failed ? never leave a partial behind
+             try:
+                 os.remove(tmp)
+             except OSError:
+                 pass
+     return final
+
+
+ @app.get("/api/source/{video_id}")
+ def get_source(video_id: str, request: Request):
+     """Stream a video for in-UI annotation (P8), Range-capable so the browser can scrub. Serves
a
+     frame-preserving PROXY (STUDIO_MEDIA_LIFECYCLE.md D4 ? downscaled + seekable, labels still
keyed
+     to the master's MD5) once one exists; otherwise serves the source and WARMS a proxy in the
-     background for next time ? non-blocking, so the first load is never delayed. Proxy warming

```

```

is
-   LOCAL-source only (remote-source proxying is a follow-up)."""
+   background for next time ? non-blocking, so the first load is never delayed. A REMOTE
source is
+   downloaded ONCE into ``<output>/source_cache/<video_id>/`` and every later request (each
browser
+   Range request hits this URL anew) streams that local copy ? never a re-download (R1-03)."""
    rec = request.app.state.db.get_video(video_id)
    if not rec or not rec.get("filepath"):
        raise HTTPException(status_code=404, detail="Video not found.")
@@ -641,20 +707,21 @@ def get_source(video_id: str, request: Request):
    if proxy_path and os.path.isfile(proxy_path):
        return FileResponse(proxy_path, media_type="video/mp4")

-   # 2) Otherwise resolve + serve the source.
+   # 2) Otherwise resolve + serve the source (remote ? the reused source-cache copy).
    try:
        local = storage.acquire_local_input(rec["filepath"], getattr(cfg, "storage", None))
        is_local = storage.input_is_local(rec["filepath"], getattr(cfg, "storage", None))
        except ValueError as e: # unsupported scheme etc. ? a clean client error, never a 500
+        local = _acquire_source_for_serving(video_id, rec["filepath"], cfg)
+    except HTTPException:
+        raise # already a clean client answer (e.g. the 507 free-space guard)
+    except ValueError as e: # unsupported scheme / unsafe id ? a clean client error, never a
500
        raise HTTPException(status_code=400, detail=str(e)) from e
    except Exception as e: # noqa: BLE001 - a fetch miss is a bad-gateway, surfaced (not
swallowed)
        raise HTTPException(status_code=502, detail=f"Could not resolve the source video: {e}")
from e
    if not os.path.isfile(local):
        raise HTTPException(status_code=404, detail="Source video file not found.")

-   # 3) Warm a proxy in the background for next time (local source only).
-   if is_local:
-       threading.Thread(target=_warm_annotation_proxy, args=(video_id, local, cfg),
daemon=True).start()
+   # 3) Warm a proxy in the background for next time. Remote sources now warm too ? the
+   #   source-cache copy is a plain local file (closes the 'remote-source proxying' follow-
up).
+   threading.Thread(target=_warm_annotation_proxy, args=(video_id, local, cfg),
daemon=True).start()

    media = "video/quicktime" if local.lower().endswith(".mov") else "video/mp4"
    return FileResponse(local, media_type=media)
diff --git a/tests/test_api_annotations.py b/tests/test_api_annotations.py
index c212810..cfe96d8 100644
--- a/tests/test_api_annotations.py
+++ b/tests/test_api_annotations.py
@@ -2,6 +2,9 @@
GET /api/source/{video_id} (serve a video's source for annotation) and
POST /api/annotations (persist finished rally labels as the byte-verbatim annotator CSV)."""

+import threading
+import time
+
import pytest

pytest.importorskip("httpx") # TestClient needs httpx; skip cleanly where absent
@@ -10,6 +13,7 @@
import backend.main as m
from backend.config.models import AppConfig
from backend.infrastructure.database import Database
+from backend.storage.ref import StorageStat

```

```

class _InlineExecutor:
@@ -64,6 +68,151 @@ def test_source_serves_the_proxy_when_one_is_ready(tmp_path, monkeypatch):
    assert r.content == b"PROXY-bytes" # proxy wins over source

+# ---- GET /api/source/{video_id} ? REMOTE-backed sources (R1-03: download ONCE, reuse) ----
+_REMOTE = b"remote-bytes"
+
+
+
+class _StubBackend:
+    """A stat-capable storage stub so the free-space guard can size the object without any
+    SDK."""
+
+    def __init__(self, size=None):
+        self._size = len(_REMOTE) if size is None else size
+
+    def stat(self, ref):
+        return StorageStat(size=self._size)
+
+
+
+def _counting_fetch(counter, payload=_REMOTE, delay=0.0):
+    """A storage.fetch double that counts downloads and writes ``payload`` to ``dst``."""
+
+    def _fetch(uri, dst=None, *, config=None, overwrite=False):
+        counter["n"] += 1
+        counter["uri"] = uri
+        if delay:
+            time.sleep(delay)
+        with open(dst, "wb") as fh:
+            fh.write(payload)
+        return str(dst)
+
+    return _fetch
+
+
+
+def test_source_remote_is_fetched_once_and_reused(tmp_path, monkeypatch):
+    """R1-03: FileResponse serves Range, so a scrubbing browser hits this URL many times ? the
+    remote object must be downloaded ONCE into the source cache and served from there after."""
+    client, db = _client(tmp_path, monkeypatch)
+    db.add_video("beefcafe", "gs://bucket/videos/match.mp4", "processed", [])
+    counter = {"n": 0}
+    monkeypatch.setattr(m.storage, "fetch", _counting_fetch(counter))
+    monkeypatch.setattr(m.storage, "get_storage", lambda *a, **k: _StubBackend())
+
+    r1 = client.get("/api/source/beefcafe")
+    r2 = client.get("/api/source/beefcafe", headers={"Range": "bytes=0-3"})
+    assert r1.status_code == 200 and r1.content == _REMOTE
+    assert r2.status_code == 206 and r2.content == _REMOTE[:4] # Range served from the cache
+    assert counter["n"] == 1 # ONE download total, both requests answered
+    assert counter["uri"] == "gs://bucket/videos/match.mp4"
+    cached = tmp_path / "source_cache" / "beefcafe" / "match.mp4"
+    assert cached.read_bytes() == _REMOTE # cached under output_base, keyed by video_id
+
+
+
+def test_source_local_path_stays_identity_passthrough(tmp_path, monkeypatch):
+    """A local-path video is served straight off its recorded path ? no fetch, no cache dir."""
+    client, db = _client(tmp_path, monkeypatch)
+    vid = tmp_path / "match.mp4"
+    vid.write_bytes(b"local-bytes")
+    db.add_video("localvid", str(vid), "processed", [])
+
+    def _boom(*a, **k):
+        raise AssertionError("fetch must not be called for a local source")
+
+    monkeypatch.setattr(m.storage, "fetch", _boom)

```

```

+
+     r = client.get("/api/source/localvid")
+     assert r.status_code == 200 and r.content == b"local-bytes"
+     assert not (tmp_path / "source_cache").exists() # nothing was copied or cached
+
+
+def test_source_remote_stale_partial_is_never_served(tmp_path, monkeypatch):
+    """A crashed/in-flight download leaves only ``*.partial`` names; the endpoint must never
+    serve one ? it re-fetches and atomically publishes the complete copy."""
+    client, db = _client(tmp_path, monkeypatch)
+    db.add_video("cafe0123", "gs://bucket/videos/match.mp4", "processed", [])
+    vid_dir = tmp_path / "source_cache" / "cafe0123"
+    vid_dir.mkdir(parents=True)
+    (vid_dir / ".match.mp4.deadbeef.partial").write_bytes(b"TRUNCATED")
+    counter = {"n": 0}
+    monkeypatch.setattr(m.storage, "fetch", _counting_fetch(counter))
+    monkeypatch.setattr(m.storage, "get_storage", lambda *a, **k: _StubBackend())
+
+    r = client.get("/api/source/cafe0123")
+    assert r.status_code == 200
+    assert r.content == _REMOTE # the complete fetch ? never the truncated partial
+    assert counter["n"] == 1
+    assert (vid_dir / "match.mp4").read_bytes() == _REMOTE
+
+
+def test_source_remote_fetch_failure_is_502_and_leaves_no_partial(tmp_path, monkeypatch):
+    """A failed download surfaces as 502 and cleans its partial, so a retry starts clean."""
+    client, db = _client(tmp_path, monkeypatch)
+    db.add_video("fail0123", "gs://bucket/videos/match.mp4", "processed", [])
+    monkeypatch.setattr(m.storage, "get_storage", lambda *a, **k: _StubBackend())
+
+    def _broken(*a, **k):
+        raise RuntimeError("bucket unreachable")
+
+    monkeypatch.setattr(m.storage, "fetch", _broken)
+    assert client.get("/api/source/fail0123").status_code == 502
+    vid_dir = tmp_path / "source_cache" / "fail0123"
+    assert not any(vid_dir.iterdir()) # no partial (or anything else) left behind
+
+    counter = {"n": 0}
+    monkeypatch.setattr(m.storage, "fetch", _counting_fetch(counter))
+    r = client.get("/api/source/fail0123") # the retry succeeds and caches normally
+    assert r.status_code == 200 and r.content == _REMOTE and counter["n"] == 1
+
+
+def test_source_remote_download_is_free_space_guarded(tmp_path, monkeypatch):
+    """D11: an object the disk can't hold is refused with 507 BEFORE any bytes are pulled."""
+    client, db = _client(tmp_path, monkeypatch)
+    db.add_video("big00001", "gs://bucket/videos/huge.mp4", "processed", [])
+    monkeypatch.setattr( # ~1 EB ? larger than any test box's free space
+        m.storage, "get_storage", lambda *a, **k: _StubBackend(size=10**18)
+    )
+
+    def _boom(*a, **k):
+        raise AssertionError("must not fetch when the disk cannot hold the object")
+
+    monkeypatch.setattr(m.storage, "fetch", _boom)
+    assert client.get("/api/source/big00001").status_code == 507
+
+
+def test_source_concurrent_first_requests_download_once(tmp_path, monkeypatch):
+    """Two racing first requests serialize on the per-video lock: ONE download, both served."""
+    cfg = AppConfig()
+    cfg.output.output_dir = str(tmp_path)
+    counter = {"n": 0}

```

```

+ monkeypatch.setattr(m.storage, "fetch", _counting_fetch(counter, delay=0.2))
+ monkeypatch.setattr(m.storage, "get_storage", lambda *a, **k: _StubBackend())
+
+ results = []
+
+ def _hit():
+     results.append(
+         m._acquire_source_for_serving("race0001", "gs://bucket/videos/match.mp4", cfg)
+     )
+
+ threads = [threading.Thread(target=_hit) for _ in range(2)]
+ for t in threads:
+     t.start()
+ for t in threads:
+     t.join()
+ assert counter["n"] == 1 # the loser waited, re-checked, and reused the winner's copy
+ assert len(results) == 2 and results[0] == results[1]
+ with open(results[0], "rb") as fh:
+     assert fh.read() == _REMOTE
+
+
+ # ---- POST /api/annotations ----
+ def test_annotations_persist_verbatim_csv(tmp_path, monkeypatch):
+     client, _ = _client(tmp_path, monkeypatch)

```

5. assistant (2026-07-02T04:57:53.825Z)

The diff touches only two files. Let me verify the file list and then read the worktree code for context.

6. user (2026-07-02T04:57:58.548Z)

```

backend/main.py
tests/test_api_annotations.py
{"base":"master","commits":["fix(api): /api/source downloads a remote source ONCE into a
reusable ?"],"head":"fix/api-source-remote-cache"}

```

7. user (2026-07-02T04:57:58.849Z)

```

924f9a3 fix(api): /api/source downloads a remote source ONCE into a reusable cache (R1-03)
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)

```

8. user (2026-07-02T04:58:09.673Z)

```

1:import argparse
2:import csv
3:import json
4:import logging
5:import math
6:import os
7:import shutil
8:import socket
9:import sys
10:import tempfile
11:import threading
12:import uuid
13:import webbrowser
14:from importlib.metadata import PackageNotFoundError
15:from importlib.metadata import version as _pkg_version
16:from urllib.parse import quote
18:import uvicorn
19:from dotenv import load_dotenv
25:from backend.utils.app_home import env_file_to_load, resolve_output_dir

```



```

36:from typing import Any, Dict, List, Optional, Tuple
38:from fastapi import FastAPI, HTTPException, Request
39:from fastapi.middleware.cors import CORSMiddleware
40:from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
41:from fastapi.staticfiles import StaticFiles
42:from pydantic import BaseModel
43:from starlette.middleware.trustedhost import TrustedHostMiddleware
45:from backend import (
49:from backend.api import (
52:from backend.config import (
59:from backend.config import ( # new typed config
62:from backend.infrastructure.database import Database
63:from backend.pipeline.segmenters.base import segmenter_registry
64:from backend.pipeline.stitcher.compiler import VideoCompiler
65:from backend.pipeline.stitcher.watermark_i18n import localize_watermark
66:from backend.pipeline.validators.base import registry
67:from backend.providers import provider_registry
68:from backend.reporting import emit_indexer_report
69:from backend.results import Failure # standardized error/result contract
70:from backend.storage.capabilities import (
73:from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
77:from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
78:from backend.utils.hashing import compute_video_id # canonical file-identity hashing
79:from backend.utils.paths import (
85:from backend.utils.redact import (
88:from backend.utils.single_instance import acquire_lock # P0e: one instance per database
292:from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable on
backend.main)
764:def _storage_settings(cfg) -> Dict[str, Any]:
1551:from backend.api import uploads as uploads_api

```

9. user (2026-07-02T04:58:11.003Z)

```

598-
599-def _proxies_dir(cfg) -> str:
600-    return os.path.join(output_base(cfg), "proxies")
601-
602-
603:def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
604-    """Background, best-effort: generate the annotation proxy ONCE so subsequent /api/source
loads
605-    serve the lighter, seekable copy. Deduped; never blocks the request; the source keeps
serving if
606-    it fails."""
607-    with _proxy_lock:
608-        if video_id in _proxy_inflight:
609-            return
610-        _proxy_inflight.add(video_id)
611-    try:
612-        from backend.utils.annotation_proxy import make_annotation_proxy, needs_proxy
613-        if not needs_proxy(src_local):
614-            return # already light enough ? no proxy worth making
615-        make_annotation_proxy(src_local, os.path.join(_proxies_dir(cfg), f"{video_id}.mp4"))
616-    except Exception as e: # noqa: BLE001 - warming is best-effort
617-        logger.warning("proxy: warm failed for %s: %s", video_id, e)
618-    finally:
619-        with _proxy_lock:
620-            _proxy_inflight.discard(video_id)
621-
622-
623-# Per-video locks so concurrent FIRST requests for a cold remote source serialize on ONE
download
624-# (the losers re-check and reuse the winner's copy) instead of each pulling the full object.
625-_source_fetch_locks: "dict[str, threading.Lock]" = {}
626-_source_fetch_locks_guard = threading.Lock()

```

```

627-
628-
629-def _source_cache_dir(cfg) -> str:
630-    """Where /api/source caches fetched REMOTE sources ? under the guarded local output
631-    root."""
632-
633-    return os.path.join(output_base(cfg), "source_cache")
634-
635-def _acquire_source_for_serving(video_id: str, raw_ref: str, cfg) -> str:
636-    """Resolve a video's recorded filepath to the LOCAL file ``/api/source`` streams.
637-
638-    A local path is an identity passthrough (no copy, no temp ? byte-identical to serving
639-    the
640-    recorded path directly). A REMOTE ref (``gs://``/``s3://``/``gdrive://``?) is fetched
641-    ONCE into
642-    ``<output>/source_cache/<video_id>/<name>`` and REUSED: ``FileResponse`` serves Range,
643-    so one
644-    scrubbing ``<video>`` tag issues many independent requests to this URL ? before this
645-    cache each
646-    request re-downloaded the full object into a fresh, never-deleted temp dir (a cloud-
647-    egress bill
648-    plus a disk-fill of the appliance; audit R1-03).
649-
650-    Concurrency + crash safety: the download goes to a unique ``*.partial`` name inside the
651-    cache
652-    dir and is published with an atomic ``os.replace`` ? a truncated file can never sit at
653-    the
654-    served name, and a failed fetch leaves nothing behind. Racing first requests serialize
655-    on a
656-    per-video lock. The download is free-space guarded (D11) when the backend can ``stat``
657-    the
658-    object's size; the cached copy is a regenerable intermediate (safe to delete any
659-    time)."""
660-
661-    settings = _storage_settings(cfg)
662-    ref = storage.resolve_input_ref(raw_ref, settings)
663-    if ref.backend == "local":
664-        return ref.key # identity ? today's local behaviour, byte-for-byte
665-
666-    vid_dir = os.path.join(_source_cache_dir(cfg), safe_output_filename(video_id))
667-    try:
668-        fname = safe_output_filename(ref.name or "in.mp4")
669-    except ValueError: # a hostile/odd remote object name never dictates the local filename
670-        fname = "in.mp4"
671-    final = os.path.join(vid_dir, fname)
672-    if os.path.isfile(final):
673-        return final # cache hit ? no re-download, whatever the browser's Range pattern
674-
675-    with _source_fetch_locks_guard:

```

10. user (2026-07-02T04:58:19.818Z)

```

__init__.py
__pycache__
base.py
capabilities.py
gcs.py
gdrive.py
gdrive_api.py
local.py
ref.py
s3.py

```

11. user (2026-07-02T04:58:21.710Z)

<persisted-output>

Output too large (30.8KB). Full output saved to: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\tool-results\toolu_01PA7HjzNvnCWxTrn7jegdMt.txt

Preview (first 2KB):

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-source-cache\backend\storage\gdrive.py:138:     def stat(self, ref) -> StorageStat:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-source-cache\backend\storage\gdrive.py:139-         self._require_present(ref)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-source-cache\backend\storage\gdrive.py:140-         return self._local.stat(self._local_ref(ref))
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-source-cache\backend\storage\gdrive.py:141-
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-source-cache\backend\storage\gdrive.py:142-     def list(self, prefix) -> Iterator[StorageRef]:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-source-cache\backend\storage\gdrive.py:143-         for r in self._local.list(self._local_ref(prefix)):
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-source-cache\backend\storage\gdrive.py:144-             uri = self._to_gdrive_uri(r.key)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-source-cache\backend\storage\gdrive.py:145-             yield StorageRef("gdrive", "",
uri[len("gdrive://") :], uri)
...
</persisted-output>
```

12. assistant (2026-07-02T04:58:22.197Z)

You've hit your session limit · resets 1:10pm (Asia/Calcutta)